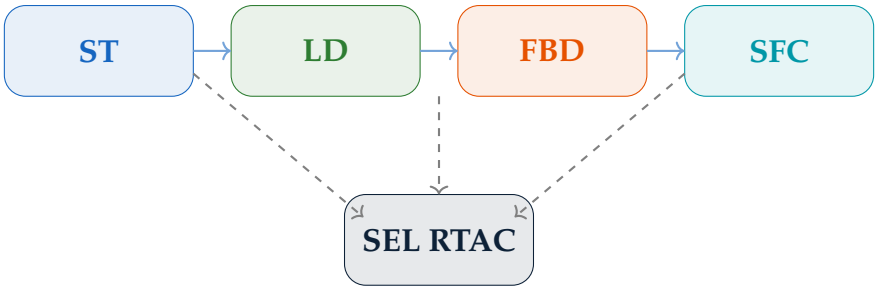
IEC 61131-3 Programming

for Power System RTACs

Structured Text, Ladder Diagram, FBD, SFC, and SEL RTAC Implementation

Ricardo Barbosa

2026



Contents

Chapter 1

IEC 61131-3 Overview

1.1 History and Purpose

IEC 61131-3 was first published in 1993 by the International Electrotechnical Commission Technical Committee 65 (Industrial-Process Measurement, Control, and Automation). Before its publication, every PLC vendor shipped proprietary programming software and syntax. A Siemens programmer could not read Allen-Bradley code. A Modicon programmer could not read GE code.

The standard defined five programming languages for industrial controllers, with the explicit goal of portability across vendor platforms. Portability in practice is imperfect — every vendor implements extensions — but the foundational knowledge transfers cleanly.

Key Concept

The five IEC 61131-3 languages:

- **Structured Text (ST)** — Pascal-derived text language. The most expressive and widely used for complex logic.
- **Ladder Diagram (LD)** — Graphical relay logic. Electricians can read it without software training.
- **Function Block Diagram (FBD)** — Graphical data-flow. Signals flow left to right through connected blocks.
- **Sequential Function Chart (SFC)** — State machine / flowchart language. Best for sequential process control.
- **Instruction List (IL)** — Assembly-like. **Deprecated in the 3rd edition (2013). Do not use.**

1.2 Third Edition (2013) Changes

The third edition of IEC 61131-3 formally deprecated Instruction List and added object-oriented extensions to FUNCTION_BLOCK: methods, properties, and inheritance. These OOP features are supported in CODESYS and some other modern IDEs. Support in ACSELERATOR RTAC depends on firmware version.

1.3 Why It Matters for RTAC Programming

The SEL RTAC executes IEC 61131-3 programs on a deterministic real-time kernel. Scan periods as fast as 1 ms are supported. When you write Structured Text in ACSELERATOR RTAC, you are writing code that will execute at real-time rates alongside protocol stacks managing DNP3 communications, IEC 61850 GOOSE publishing, and relay I/O.

Platform-specific knowledge matters, but the foundational IEC 61131-3 concepts — data types, variable scoping, POU structure, task configuration — are identical across all compliant platforms.

Chapter 2

Data Types and Variables

2.1 Elementary Data Types

Type	Size	Range / Notes
`BOOL`	1 bit	`TRUE` or `FALSE`
`BYTE`	8 bits	0–255. Unsigned. Bit manipulation.
`WORD`	16 bits	0–65535. Unsigned.
`DWORD`	32 bits	0–4294967295. Unsigned.
`INT`	16 bits	-32768 to $+32767$. Signed.
`DINT`	32 bits	$-2{,}147{,}483{,}648$ to $+2{,}147{,}483{,}647$. Signed.
`UINT`	16 bits	0–65535. Unsigned integer.
`REAL`	32 bits	IEEE 754 single precision. $\approx$7 significant digits.
`LREAL`	64 bits	IEEE 754 double precision. $\approx$15 significant digits.
`TIME`	32 bits	Duration. Literal: `T#5s`, `T#500ms`.
`DATE`	32 bits	Calendar date. Literal: `D#2026-05-09`.
`STRING`	variable	Fixed-length. Default `STRING[80]`.

2.2 Variable Declaration Sections

Section	Scope	Writable by caller?
`VAR`	Local to POU	No — internal only
`VAR_INPUT`	Set by caller, read by POU	Yes
`VAR_OUTPUT`	Written by POU, read by caller	No
`VAR_IN_OUT`	Passed by reference (bidirectional)	Yes
`VAR_GLOBAL`	Shared across all POUs	Any POU with access
`VAR RETAIN`	Survives power cycle (NVM)	Same as `VAR`
`VAR CONSTANT`	Read-only within POU	Never

WARNING

`VAR RETAIN` variables are stored in non-volatile memory. A `RETAIN` variable survives power loss and watchdog trips. Failure to mark setpoints and accumulated values as `RETAIN` means they reset to initial values after any power event — potentially at wrong process setpoints.

Chapter 3

Structured Text (ST)

3.1 Syntax Fundamentals

Structured Text uses Pascal-derived syntax. Every statement ends with a semicolon. Assignment is `:=`. Comparison uses `=`, `<>`, `>`, `<`, `>=`, `<=`.

```
(* Block comment *)
// Line comment

PROGRAM MotorInterlock
VAR
    bRunCmd      : BOOL;
    rVoltage     : REAL;
    nCount       : INT := 0;
END_VAR

bRunCmd  := TRUE;           (* Assignment operator := *)
nCount   := nCount + 1;
rVoltage := 13.8;

IF rVoltage > 14.4 THEN     (* Comparison uses = *)
    bRunCmd := FALSE;
ELSIF rVoltage < 12.0 THEN
    bRunCmd := FALSE;
ELSE
    bRunCmd := TRUE;
END_IF;
END_PROGRAM
```

3.2 CASE Statement

```
CASE nState OF
    0: (* IDLE *)
        IF bStartCmd AND bPermissive THEN
            nState := 1;
        END_IF;
    1: (* RUNNING *)
        bMotorRun := TRUE;
        IF bStopCmd THEN
            nState    := 0;
            bMotorRun := FALSE;
        END_IF;
    ELSE
        nState := 0;  (* Catch unknown states *)
END_CASE;
```

3.3 Motor Interlock Example

The following Structured Text implements a complete motor interlock with start/stop latching, emergency stop, and fault latch logic.

```
(* Motor Interlock     complete ST implementation *)
FUNCTION_BLOCK FB_MotorInterlock
VAR_INPUT
    bStart         : BOOL;
    bStop          : BOOL;
    bEmergencyStop : BOOL;
    bOverloadRelay : BOOL;
    bFaultReset    : BOOL;
END_VAR
VAR_OUTPUT
    bMotorRun      : BOOL;
    bFaultLamp     : BOOL;
    bReadyLamp     : BOOL;
END_VAR
VAR
    nState         : INT := 0;
    bFaultLatch    : BOOL := FALSE;
    tStartFail     : TON;
END_VAR

CASE nState OF
    0: (* IDLE *)
        bMotorRun  := FALSE;
```

```
        bReadyLamp := NOT bFaultLatch;
        tStartFail(IN := FALSE);
        IF bStart AND NOT bEmergencyStop AND NOT bFaultLatch THEN
            nState := 1;
        END_IF;

    1: (* RUNNING *)
        bMotorRun := TRUE;
        (* Start failure timer     trip if not confirmed in 5s *)
        tStartFail(IN := TRUE, PT := T#5s);
        IF tStartFail.Q THEN
            bFaultLatch := TRUE;
            nState := 2;
        END_IF;
        IF bStop OR bEmergencyStop THEN
            nState    := 0;
            bMotorRun := FALSE;
            tStartFail(IN := FALSE);
        END_IF;
        IF bOverloadRelay THEN
            bFaultLatch := TRUE;
            nState      := 2;
        END_IF;

    2: (* FAULT *)
        bMotorRun  := FALSE;
        bFaultLamp := TRUE;
        tStartFail(IN := FALSE);
        IF bFaultReset AND NOT bOverloadRelay THEN
            bFaultLatch := FALSE;
            bFaultLamp  := FALSE;
            nState      := 0;
        END_IF;
END_CASE;
END_FUNCTION_BLOCK
```

Chapter 4

Ladder Diagram (LD)

4.1 Elements and Conventions

Ladder Diagram represents logic as horizontal rungs between two vertical power rails. The left rail is logical power (always TRUE). Each rung evaluates left to right. If the logical path is TRUE, the output coil on the right activates.

Key elements: normally open contact (–| |–), normally closed contact (–|/|–), output coil (–( )–), set coil (–(S)–), reset coil (–(R)–), TON/TOF/TP timer blocks, CTU/CTD counter blocks.

4.2 Timer Function Blocks

Three standard timer FBs are defined in IEC 61131-3:

- **TON** (Timer On-Delay): Output `Q` goes TRUE after input `IN` has been TRUE for duration `PT`.
- **TOF** (Timer Off-Delay): Output `Q` goes TRUE immediately when `IN` goes TRUE, stays TRUE for `PT` after `IN` goes FALSE.
- **TP** (Pulse): Output `Q` goes TRUE for exactly `PT` when `IN` transitions TRUE.

Field Gotcha

Each timer FB must have its own declared instance. Using the same `TON` instance for two logically separate timing functions produces incorrect behavior that is difficult to diagnose. One timer — one instance. Always.

Chapter 5

Function Block Diagram (FBD)

5.1 Data Flow Programming

FBD is graphical. Rectangular blocks have named inputs on the left and outputs on the right. Wires connect block outputs to block inputs. Signals flow strictly left to right. Execution order is determined by data dependency, not scan order.

Standard logic blocks: AND, OR, NOT, XOR, ADD, SUB, MUL, DIV, GT, LT, EQ. FB instances (TON, CTU, custom FBs) can also be placed as blocks in FBD.

5.2 PID Control

FBD is the preferred language for PID control loops. The `CTRL_PID` function block takes setpoint `W` and process variable `X` as inputs and outputs control signal `Y`. The wiring topology makes the feedback loop structure immediately visible in the diagram.

Chapter 6

Sequential Function Chart (SFC)

6.1 Steps and Transitions

SFC describes sequential behavior as a directed graph. **Steps** (rectangles) represent states. One or more steps are active at any time. An **initial step** (double border) is active at startup. **Transitions** (horizontal lines with conditions) specify when execution moves from one step to the next.

6.2 Action Qualifiers

Each action on a step has a qualifier: **N** (non-stored — active while step active), **S** (set — latches TRUE on step activation), **R** (reset — resets a Set action), **P** (pulse — one scan only on activation).

WARNING

A Set (S) action remains active after its step deactivates. It must be explicitly Reset (R) by a downstream step. Forgetting to add the matching Reset is the most common SFC programming error.

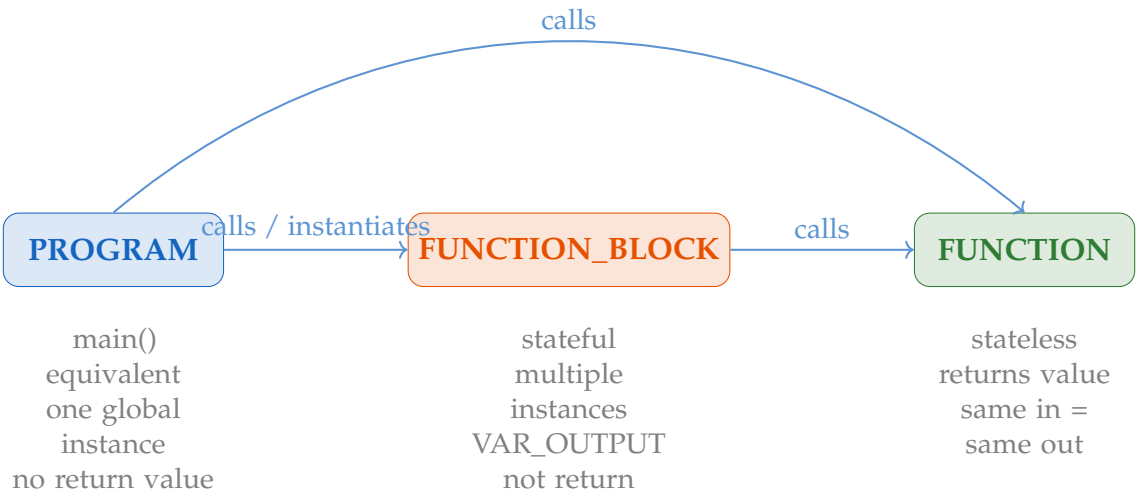
6.3 Parallel Sequences

A double horizontal line (AND divergence) activates multiple branches simultaneously. A double horizontal line convergence waits for all parallel branches to complete before proceeding.

Chapter 7

POUs: Program, Function, Function Block

7.1 POU Hierarchy



7.2 FUNCTION vs. FUNCTION_BLOCK

The critical distinction: a **FUNCTION** is stateless — its local variables reinitialize on every call. A **FUNCTION_BLOCK** is stateful — its internal variables persist between calls. To use a FUNCTION_BLOCK, you declare a named instance; each instance has independent state.

Key Concept

Use `FUNCTION` for pure computation (scaling, CRC, unit conversion). Use `FUNCTION_BLOCK` for anything that needs memory between scans: timers, counters, state machines, PID controllers.

7.3 No Recursion

IEC 61131-3 explicitly prohibits recursive calls. A FUNCTION or FUNCTION_BLOCK cannot call itself, directly or indirectly. The compiler detects and rejects recursive call chains. Use iteration instead.

Chapter 8

IEC 61131-3 on the SEL RTAC

8.1 ACSELERATOR RTAC IDE

ACSELERATOR RTAC is SEL's configuration and programming IDE for the SEL-3505, SEL-3530, SEL-3555, and related RTAC family devices. The IDE integrates IEC 61131-3 programming with protocol configuration (DNP3, IEC 61850, Modbus) in a single project file.

8.2 Task Configuration

Tasks are configured with a scan period and priority. The IEC 61131-3 runtime calls each task's assigned PROGRAM at the specified interval.

Scan Period	Typical Use	Caution
1 ms	Fast protection, time-critical interlocks	Very tight budget
10 ms	Control logic, analog processing	Common for most RTAC logic
100 ms	Status updates, protocol refresh	Comfortable budget
1 s	Logging, diagnostics	No time pressure

8.3 Data Mapping: Protocol to ST Variables

The RTAC data model bridges protocol data and IEC 61131-3 variables. Protocol clients (DNP3, IEC 61850, Modbus) populate the data model. ACSELERATOR maps data model points to global variables. ST programs read and write these globals each scan.

Field Gotcha

A global variable not mapped to a data model point always reads its initial value (0 or FALSE). If your analog reading is stuck at zero despite the remote device communicating, check the ACSELERATOR data mapping screen first.

Chapter 9

Debugging and Troubleshooting

9.1 Online Monitoring

When connected to a live RTAC from ACSELERATOR, the watch window shows current variable values updated each scan. Variable forcing temporarily overrides program logic. **Always clear all forces before disconnecting from a production device.**

9.2 Scan Time Overruns and Watchdog Trips

If a task executes longer than its scan period, the RTAC logs a scan overrun event. Repeated overruns trigger a watchdog trip, restarting the IEC 61131-3 runtime. Non-RETAIN variables reset to initial values. RETAIN variables survive.

Common overrun causes: unbounded loops, excessive iteration counts, string operations in fast tasks, misconfigured scan period.

9.3 Common RTAC IEC 61131-3 Bugs

1. State variable not initialized correctly after watchdog trip
2. Timer instance shared between two logical uses
3. Type mismatch without explicit conversion
4. Global variable not mapped to data model point
5. Scan period too aggressive for logic complexity

Chapter 10

Lab and Practice

10.1 Free Tools

- **CODESYS** (https://www.codesys.com) — Full IEC 61131-3 IDE with built-in soft PLC simulator. Free for development. Supports all five languages, breakpoints, watch windows.
- **OpenPLC Runtime** (https://openplcproject.com) — Open-source IEC 61131-3 runtime for Linux, Windows, Raspberry Pi. Free.
- **BEREMIZ** (https://beremiz.org) — Python-based open source IEC 61131-3 IDE.

10.2 Suggested Exercises

1. Motor interlock in Ladder Diagram
2. Same logic in Structured Text (CASE state machine)
3. Encapsulate as `FB_MotorInterlock`, instantiate three times
4. Analog scaling `FUNCTION` with type-safe conversion
5. Substation startup sequence in SFC (or ST CASE equivalent)

Pro Tip

Complete all exercises in CODESYS with simulation before touching ACSELERATOR RTAC. The logic validation work is identical in both environments. CODESYS gives you breakpoints.

Appendix A

Quick Reference

A.1 ST Operators Summary

Category	Operators	Notes
Assignment	`:=`	Not `=`
Arithmetic	`+ - * / MOD`	Standard math
Comparison	`= <> > < >= <=`	Returns `BOOL`
Logical	`AND OR NOT XOR`	On `BOOL` operands
Bitwise	`AND OR NOT XOR`	On `BYTE/WORD/DWORD`
Shift	`SHL(x,n)` `SHR(x,n)`	Shift left/right by n bits

A.2 Professional Certification Context

No individual exam-based credential currently exists specifically for IEC 61131-3. Practitioners in industrial automation who work with IEC 61131-3 typically pursue:

- **ISA CCST** (Certified Control Systems Technician) — covers industrial communications protocols, PID control, SCADA integration, and field instrumentation. Levels I, II, and III based on experience. See isa.org/certification/ccst.
- **ISA CAP** (Certified Automation Professional) — broader engineering credential covering process control, automation systems, and integration. Requires engineering background. See isa.org/certification/cap.
- **Vendor-Specific Context**:

- For Modbus: Inductive Automation Core/Gold certification tests Modbus connectivity in Ignition context.
- For DNP3: Triangle Microworks and SEL University offer training (PDHs, not credentials).
- For OPC UA: MatrikonOPC OPCSI is a vendor-sponsored credential for OPC systems integrators.
- For IEC 61131-3: Vendor-specific certs (Rockwell, Siemens, Beckhoff) test platform-specific implementations. SEL University APP RTAC ADV-1 covers IEC 61131-3 in RTAC context.
- For RTAC: SEL University APP RTAC course awards PDHs. No formal credential exam.